InfoMed: relatório de progresso do desenvolvimento

Documento de passagem para quem escreve o texto do TCC. Posição em 20/10/2026.

1. Resumo para quem tem pouco tempo

O **InfoMed** é um aplicativo de saúde pública digital que reúne consulta de medicamentos nos postos de saúde, localização das unidades, agendamento de consultas e lembretes de medicação. O protótipo de 20 telas foi feito no Figma Make e, nesta fase, foi transformado em um projeto de software organizado, versionado no GitHub e conectado a um banco de dados real na nuvem.

O que já existe e funciona:

- Projeto rodando localmente, limpo e versionado no GitHub.
- Camada de arquitetura em camadas, com o padrão de projeto **Repository** aplicado de verdade no código.
- Banco de dados **PostgreSQL no Supabase** com 9 tabelas, dados de exemplo e segurança por usuário (**RLS**).
- Tela de **Medicamentos** lendo dados do banco, com busca.
- **Login e cadastro** com aceite obrigatório da **LGPD** (código entregue e testado automaticamente; falta confirmar no ambiente real, ver seção 11).
- **Documentação técnica** em `/docs`: 8 decisões de arquitetura (ADRs), diagramas, fichas de funcionalidades e plano de testes de segurança.

O que falta (resumo): postos reais de Santa Bárbara d'Oeste com mapa, agendamento, lembretes e notificações, funcionalidade de IA, pipeline de integração contínua, dashboard de BI, pesquisa de usabilidade e o texto final. Detalhes na seção 12.

2. Contexto do projeto

Item	Situação
Tema	InfoMed: sistema de informação de medicamentos e serviços de saúde pública
Cidade-alvo	**Santa Bárbara d'Oeste (SP)**. Os postos e médicos do banco ainda são fictícios e serão substituídos por dados reais da cidade

Item	Situação
Prazo	Máximo de 1 mês a partir de 20/10/2026
Divisão de trabalho	Um integrante desenvolve o código; outro escreve a documentação do TCC. Este documento é a ponte entre os dois
Orientação técnica	Decisões de stack e arquitetura tomadas com apoio de IA (Claude) em formato de mentoria e *pair programming*
Princípio	Entregar menos, porém bem estruturado e documentado, evitando aparência de trabalho desorganizado

3. Disciplinas do semestre e o que cada uma vira no projeto

O TCC integra as matérias do 2º ano do curso de Sistemas. A estratégia foi transformar cada disciplina em um módulo concreto do InfoMed.

Disciplina	Entrega no projeto	Situação
Projeto Integrador II	O TCC em si, com o InfoMed como produto integrador	Em andamento
Modelagem de Padrões de Projetos	Repository (feito), Observer (notificações), Strategy (filtros), Factory (agendamentos), com diagramas UML	Repository feito
Programação Multi Plataforma	Aplicativo único em PWA, com possibilidade de empacotar para Android (Capacitor)	Base pronta
Computação em Nuvem	Backend e banco no Supabase (região São Paulo) e hospedagem do front	Banco feito; hospedagem pendente
Integração e Entrega Contínua	GitHub Actions com verificação de tipos, testes, build e deploy	Pendente
Segurança e Defesa Cibernética	Autenticação, RLS, gestão de chaves, LGPD	Feito (falta evidência real)

Disciplina	Entrega no projeto	Situação
Inteligência Artificial e Aprendizado de Máquina	Uma funcionalidade simples: sugerir a especialidade a partir do sintoma	Pendente
Business Intelligence e Big Data	Dashboard sobre postos, medicamentos e consultas	Pendente
Sistemas Distribuídos aplicados a IoT	Caixa de remédios simulada (opcional)	Opcional
Estatística Aplicada	Questionário de usabilidade (SUS) com análise descritiva	Pendente
Sistemas de Informação e Tecnologias Emergentes	Capítulo de fundamentação justificando as tecnologias	Insumos nos ADRs
Língua Inglesa II	Abstract e README em inglês	Pendente

4. Escopo congelado

Para caber em um mês, o escopo foi dividido por prioridade.

- **P0 (obrigatório):** login, lista e detalhe de medicamentos, lista e detalhe de postos com mapa, agendamento (especialidade, médico, data e hora, confirmação), lembretes e notificações.
- **P1 (se der tempo):** histórico, cancelar consulta, perfil, filtros no mapa e a funcionalidade de IA.
- **P2 (só se sobrar):** IoT e dashboard avançado.

5. Decisões técnicas (stack)

Camada	Escolha	Motivo principal
Front-end	React 18, Vite, TypeScript, Tailwind CSS	O Figma Make já exporta nesse formato, permitindo reaproveitar as telas
Multiplataforma	PWA, com Capacitor para Android se necessário	Um único código para web, Android e iOS
Backend e banco	Supabase (PostgreSQL, Auth, RLS, Edge Functions)	Cobre nuvem e segurança sem escrever uma API do zero

Camada	Escolha	Motivo principal
Mapa	Leaflet com OpenStreetMap	Gratuito e sem chave de API
Testes	Vitest com Testing Library	Integra-se ao Vite e roda no CI
CI/CD	GitHub Actions	O código já está no GitHub
BI	Views SQL e Looker Studio (plano B: página com gráficos)	Aproveita o banco relacional
IA	Edge Function chamando uma API de modelo de linguagem	A chave fica no servidor, nunca no navegador

6. O que foi feito, em ordem

6.1. Análise do código exportado do Figma Make

O Figma Make gerou 20 telas em React e TypeScript. A interface estava boa, mas o código tinha características de protótipo. A análise encontrou:

Achado	Impacto	Resolução
`package.json` com entradas no formato `pacote@versão`, próprias do Figma	`npm install` falhava com o erro `EINVALIDPACKAGENAME`	`package.json` reescrito e limpo
Imports com versão no nome (ex.: `lucide-react@0.487.0`) em 41 arquivos	Build falhava fora do Figma	Script `fix-imports.mjs` removeu as versões
Dependências sem uso (MUI, emotion, react-slick, react-dnd, confetti, motion, date-fns e outras)	Peso e instalação lenta	Removidas
Navegação por `useState`, sem rotas	Botão voltar e links diretos não funcionam	**Pendente** (migrar para react-router)
Dados fixos dentro de cada tela	Sem camada de dados	Resolvido para Medicamentos; demais telas pendentes

Achado	Impacto	Resolução
Uso de `any` e ausência de `tsconfig.json`	Sem verificação de tipos	`tsconfig.json` criado; erros zerados
Moldura de iPhone envolvendo o app	Inadequada em celular real	**Pendente**
Textos fixos ("Maria Silva", "Dezembro 2024", contador de notificações)	Não refletem dados reais	Nome já resolvido; demais pendentes
Sem git, testes e integração contínua	Não atende à disciplina de CI/CD	Git feito; testes iniciados; CI pendente

Também foi corrigido um erro de tipo real na tela de detalhe do medicamento (a função `navigate` não aceitava dados).

6.2. Ambiente e versionamento

- Instalação de Node.js, VS Code e Git.
- Projeto movido para `C:\projetos\InfoMed`, **fora do OneDrive**, para evitar lentidão e erros com a pasta `node_modules`.
- Repositório criado no GitHub (privado) e primeiro commit enviado.

6.3. Camada de domínio e padrão Repository

- A entidade `Medication` foi criada em `src/app/domain`.
- A interface `MedicationRepository` define o contrato (`list` e `getById`).
- Duas implementações: `MockMedicationRepository` (dados em memória) e `SupabaseMedicationRepository` (banco real).
- O arquivo `repositories/index.ts` é o **único** ponto que escolhe a implementação, conforme a existência do arquivo `.env`.
- As telas dependem apenas da interface. A troca de Mock para Supabase **não exigiu alterar nenhuma tela**, o que valida a decisão.

6.4. Banco de dados no Supabase

Projeto criado na região São Paulo, com os scripts:

Script	Função
`db/01_schema.sql`	Tabelas, índices, gatilho de perfil, função auxiliar, view e políticas de segurança
`db/02_seed.sql`	Dados de exemplo fictícios (medicamentos, unidades, estoque, especialidades, médicos)
`db/03_teste_rls.sql`	Teste de isolamento entre usuários (evidência para o TCC)
`db/04_auth.sql`	Atualiza o gatilho para gravar o aceite da LGPD no perfil

6.5. Autenticação e LGPD

- Cadastro e login por e-mail e senha com **Supabase Auth**.
- Caixa de aceite da LGPD **obrigatória** no cadastro; o momento do aceite é gravado em `profiles.lgpd_consent_at`.
- Senha com no mínimo 8 caracteres; mensagens de erro traduzidas e sem revelar detalhes internos.
- Telas internas bloqueadas para quem não está logado.
- Home e Perfil passaram a mostrar o nome do usuário logado; botão **Sair da Conta** funcionando.
- Padrão Repository aplicado também à autenticação (`AuthRepository` , com implementação Supabase e Mock).
- 4 testes automatizados cobrindo login e cadastro com LGPD.

6.6. Documentação

Criada a pasta `docs/` no repositório com ADRs, diagramas, fichas de funcionalidade e testes de segurança (lista completa na seção 10).

7. Arquitetura atual

```
flowchart TB
  U[Usuário] --> PWA[PWA React + Vite + TypeScript]
  subgraph Front-end
    PWA --> T[Telas / features]
    T --> R[Repository - interface]
    R --> M[Mock - sem .env]
    R --> S[Supabase - com .env]
  end
  S --> DB[(PostgreSQL + RLS)]
  S --> AU[Auth]
  DB --> V[Views SQL]
  GH[GitHub] --> PWA
```

Camada	Pasta	Responsabilidade
Interface	`src/app/components`	Telas e componentes visuais
Domínio	`src/app/domain`	Entidades e tipos (`Medication` , `AuthUser`)
Repositórios	`src/app/repositories`	Contratos e implementações de acesso a dados
Infraestrutura	`src/app/lib`	Conexão com o Supabase
Estado e funcionalidades	`src/app/features`	Estado compartilhado, como a autenticação

8. Banco de dados e regras de acesso

Tabelas de catálogo (leitura pública): `medications` , `health_units` , `unit_medications` (estoque por unidade), `specialties` , `doctors` .

Tabelas de dados pessoais (cada usuário só acessa o que é seu): `profiles` , `appointments` , `reminders` , `notifications` .

Tabela	Leitura	Escrita
Catálogo (5 tabelas)	Pública	Somente administrador, pelo painel
profiles	Somente o próprio usuário	O próprio usuário atualiza; criação automática no cadastro

Tabela	Leitura	Escrita
appointments	Somente o próprio usuário	O próprio usuário insere e atualiza; sem exclusão
reminders	Somente o próprio usuário	Inserir, atualizar e excluir o próprio
notifications	Somente o próprio usuário	O usuário marca como lida; a criação é do servidor

Objetos auxiliares

- View `medications_with_status`: calcula o status do medicamento (disponível, estoque baixo, indisponível) a partir do estoque de cada unidade.
- Função `booked_slots`: informa os horários já ocupados de um médico sem expor dados de outros usuários.
- Gatilho `on_auth_user_created`: cria o perfil quando o usuário se cadastra.
- Índice único que impede dois agendamentos ativos no mesmo horário com o mesmo médico.

O diagrama entidade-relacionamento completo está em `docs/diagrams/modelo-er.md`.

9. Segurança e LGPD

Controle	Como funciona
Row Level Security em todas as tabelas	Sem política que permita, nenhuma linha é acessível
Isolamento por usuário	Políticas `user_id = auth.uid()`
Prevenção de falsificação	Não é possível inserir dados em nome de outro usuário
Privilégios mínimos	O papel anônimo não acessa dados pessoais
Gestão de chaves	Somente a chave *publishable* no front-end; a chave *secret* nunca vai para o código; o arquivo `.env` fica fora do GitHub
Consentimento LGPD	Aceite obrigatório e registrado com data e hora por usuário
Senhas	Tratadas pelo Supabase Auth, nunca pelo código do projeto

Pontos futuros de LGPD: recuperação de senha, exclusão de conta e exportação dos dados (direitos do titular), além de política de privacidade completa.

10. Documentação existente no repositório (`/docs`)

Arquivo	Conteúdo
`README.md`	Índice geral e mapa disciplina para documentação
`decisions/ADR-001`	Stack do front-end
`decisions/ADR-002`	Do protótipo do Figma Make para código de produção (inclui a tabela de problemas)
`decisions/ADR-003`	Padrão Repository
`decisions/ADR-004`	Supabase como backend na nuvem
`decisions/ADR-005`	Leaflet e OpenStreetMap (a implementar)
`decisions/ADR-006`	Integração contínua com GitHub Actions (a implementar)
`decisions/ADR-007`	Variáveis de ambiente e gestão de chaves

Arquivo	Conteúdo
`decisions/ADR-008`	Autenticação e consentimento LGPD
`features/F01` a `F04`	Fichas das funcionalidades concluídas
`features/_modelo.md`	Modelo para novas fichas
`diagrams/`	Arquitetura, modelo ER, fluxo de autenticação
`security/testes-rls.md`	Controles de segurança e 8 casos de teste
`evidence/`	Pasta para prints (ainda a preencher)

Cada ADR segue o formato: contexto, decisão, alternativas consideradas, consequências e evidências. **Esse formato foi pensado para virar capítulo do TCC** com pouca adaptação.

11. Testes realizados e o que ainda precisa ser comprovado

Já realizado

- Verificação de tipos (`tsc`) sem erros e build de produção bem-sucedido.
- 4 testes automatizados passando (login com sucesso, login com campos vazios, cadastro bloqueado sem aceite da LGPD, cadastro com aceite).
- O esquema do banco e as políticas de RLS foram validados em um PostgreSQL local com `auth` simulado. Resultado: o usuário B não vê consultas nem lembretes do usuário A, não consegue inserir dados em nome do A, dois agendamentos no mesmo horário são bloqueados, e o acesso anônimo a dados pessoais é negado.
- Confirmado no ambiente real: o SQL rodou no Supabase e a tela de Medicamentos mostra dados do banco.

Ainda a comprovar no ambiente real (evidência para o TCC)

- Fluxo de login e cadastro no navegador, com conferência da tabela `profiles` (nome e `lgpd_consent_at`).
- Teste de isolamento com duas contas reais, usando `db/03_teste_rls.sql` , e preenchimento da coluna "Resultado obtido" em `docs/security/testes-rls.md` .
- Prints em `docs/evidence/` : tabelas no Supabase, políticas de RLS, telas de login e cadastro, Home com o nome do usuário, `git status` sem o arquivo `.env` .

12. Problemas enfrentados e soluções

Estes episódios rendem bom material para o capítulo de metodologia e resultados.

Problema	Causa	Solução
`npm` bloqueado no PowerShell	Política de execução de scripts do Windows	`Set-ExecutionPolicy RemoteSigned`
`package.json` não encontrado	Pasta extraída duplicada (`InfoMed` dentro de `InfoMed`)	Mover o conteúdo para o nível correto
Projeto dentro do OneDrive	Sincronização de milhares de arquivos de `node_modules`	Mover para `C:\projetos`
`EINVALIDPACKAGENAME` no `npm install`	Formato de dependências exclusivo do Figma Make	Reescrever o `package.json`
Build quebrado fora do Figma	Versões embutidas nos imports	Script de correção automática
Conflito de versão ao instalar o Vitest	A versão mais nova exige um Vite mais recente	Usar `vitest@3`, compatível com o Vite do projeto

13. Pendências e próximos passos

Imediato

1. Confirmar no ambiente real o login, o cadastro e o teste de RLS com duas contas, e coletar os prints.
2. Substituir as unidades fictícias por **UBS e unidades de saúde reais de Santa Bárbara d'Oeste**: nome, endereço, coordenadas, horário e serviços, a partir de fontes públicas (CNES, OpenStreetMap e prefeitura). Os dados precisam ser conferidos antes da apresentação, pois horários e serviços mudam. Os dados de estoque de medicamentos por unidade permanecerão como **simulados**, e isso deve ser dito no TCC.
3. Mapa com Leaflet nas telas de postos (ADR-005).

Semana 2 a 3 (conforme o cronograma)

4. Migrar a navegação para rotas (react-router).
5. Agendamento de consultas ponta a ponta, usando o padrão Factory.
6. Lembretes e notificações, usando o padrão Observer.
7. Filtros dos postos com o padrão Strategy.
8. Funcionalidade de IA por Edge Function.
9. Pipeline de CI no GitHub Actions e deploy do aplicativo (ADR-006).

10. Dashboard de BI com views SQL.
11. Pesquisa de usabilidade (SUS) com 15 a 30 pessoas.

Semana 4

12. Texto final, diagramas UML finais, apresentação e congelamento do código (somente correção de bugs).

14. Orientações para quem escreve o texto do TCC

Estrutura sugerida do documento

1. Introdução (problema, justificativa, objetivos)
2. Fundamentação teórica (saúde digital, nuvem, segurança e LGPD, IA, BI, padrões de projeto)
3. Metodologia (levantamento, prototipação, desenvolvimento ágil)
4. Desenvolvimento (requisitos, UML, arquitetura, telas, padrões aplicados, CI/CD, segurança)
5. Resultados (testes, pesquisa SUS, estatística, dashboard)
6. Conclusão e trabalhos futuros
7. Abstract e referências (ABNT)

Como aproveitar o material pronto

- Os **ADRs** alimentam os capítulos de fundamentação e desenvolvimento, pois cada um traz contexto, alternativas e consequências.
- A tabela de achados do **ADR-002** é um bom exemplo de análise crítica do uso de IA generativa na prototipação.
- Os **diagramas** estão em Mermaid, que o GitHub desenha automaticamente; podem ser exportados como imagem para o documento.
- As **fichas de funcionalidade** relacionam cada requisito à disciplina correspondente, o que ajuda a demonstrar a integração entre as matérias.
- Quando algo estiver marcado como "a implementar" ou "pendente", **não deve ser descrito no TCC como concluído**.

Pontos de honestidade que precisam constar no texto

- O protótipo de interface foi gerado com apoio de IA (Figma Make) e depois refatorado.
- Os dados de estoque de medicamentos por unidade são simulados; os dados das unidades de saúde vêm de fontes públicas e foram conferidos manualmente.
- O aplicativo não substitui atendimento médico, e a funcionalidade de IA, quando existir, deve exibir esse aviso.

15. Glossário rápido

Termo	Significado
ADR	*Architecture Decision Record*: registro curto de uma decisão técnica
Repository	Padrão de projeto que separa as telas da origem dos dados
Mock	Implementação falsa usada em desenvolvimento e testes
RLS	*Row Level Security*: regra no banco que limita as linhas visíveis para cada usuário
PWA	*Progressive Web App*: site instalável que funciona como aplicativo
Supabase	Plataforma de backend na nuvem baseada em PostgreSQL
LGPD	Lei Geral de Proteção de Dados
CI/CD	Integração e entrega contínuas: automação de testes, build e publicação
SUS	*System Usability Scale*: questionário padrão de usabilidade